

Tyler Buck
CS499
August 2nd, 2026

Enhancement Three: Databases

Artifact Description

The artifact selected for this enhancement is the Grazioso Salvare animal-shelter dashboard that I originally created for CS 340 in April 2026. The project uses Python, MongoDB, PyMongo, Pandas, and Dash to retrieve and display animal records from a MongoDB database. A reusable AnimalShelter CRUD module handles communication with the database, while the dashboard allows users to filter records by rescue category and view the results through an interactive table, pie chart, and map.

The original application supported Water Rescue, Mountain or Wilderness Rescue, Disaster or Individual Tracking, and Reset filters. These filters used combinations of breed, sex, and age requirements to identify animals that matched each training category.

Justification for Inclusion and Enhancement

I selected this artifact because it demonstrates several important database-development skills, including MongoDB queries, CRUD operations, data retrieval, database integration, and separation of responsibilities between the data-access layer and the user interface. The reusable AnimalShelter class keeps MongoDB operations outside the dashboard code, making the project more modular and maintainable.

The original artifact was also a strong candidate for enhancement because its database-access layer trusted query dictionaries without validating their contents. It also depended on hard-coded course credentials and did not include a database index designed around the dashboard's common search fields.

To complete the enhancement, I added validation that restricts read queries to approved fields, MongoDB operators, and expected value types. The enhanced module supports the operators required by the existing dashboard, including \$or, \$gte, \$lte, \$regex, and \$options, while

rejecting unsupported fields, malformed structures, and invalid values before they reach MongoDB.

I also changed the update and delete methods so they reject empty queries. This prevents an empty filter from unintentionally modifying or deleting every document through the CRUD wrapper.

The enhanced database includes a compound index named `rescue_search_idx` for the following fields:

- `animal_type`
- `sex_upon_outcome`
- `age_upon_outcome_in_weeks`

These fields were selected because they are used repeatedly in the rescue-category queries. The index supports the project's common search pattern while also demonstrating the trade-off between faster access paths and the additional storage and maintenance required by indexes.

The enhanced notebook no longer contains the original hard-coded username and password. The database connection can operate locally without authentication or accept credentials when the project is used in an authenticated environment.

Because the original course database was no longer available, I needed a working database to test the enhancement. Rather than spending time recreating the entire original dataset, I used generative AI to help create a small mock MongoDB dataset with the same fields needed by the dashboard. I reviewed and tested the generated records before using them. The mock data allowed me to verify the database enhancement and confirm that the original dashboard filters still behaved correctly.

Course Outcome Alignment

This enhancement supports Course Outcome Four because I used Python, PyMongo, MongoDB, database indexing, query validation, and automated testing to improve a working database

solution. The enhanced artifact is more portable, safer to use, and easier to test than the original version.

The enhancement also supports Course Outcome Five because I applied a security mindset to database access. I removed embedded credentials from the enhanced notebook, restricted accepted query fields and operators, validated value types, and blocked empty update and delete operations.

Course Outcome Three is supported through the design decisions and trade-offs involved in the enhancement. Restricting queries improves safety and predictability but reduces the flexibility of the CRUD module. Adding an index can support frequently used searches but also adds storage and write-maintenance costs.

The outcome-coverage plan remained focused on Outcomes Three, Four, and Five. After reviewing the original artifact, I narrowed the enhancement to query safety, database reliability, indexing, and secure configuration rather than adding unrelated features.

Reflection on the Enhancement Process

This enhancement helped me better understand the role of a database-access layer as a boundary between an application and its data. The original CRUD module successfully connected the dashboard to MongoDB, but it assumed that every query supplied by the caller was valid. Adding field, operator, and type validation showed me how a reusable module can enforce rules before database operations are performed.

I also learned that database safety involves more than protecting login credentials. Empty update or delete filters can be dangerous even when the underlying database is functioning correctly. Adding safeguards against those operations made the module more predictable and reduced the risk of unintended bulk changes.

The indexing portion of the enhancement reinforced that database performance decisions should be based on actual query patterns. I selected fields that are repeatedly used by the dashboard instead of adding indexes without a specific purpose. This also required me to consider the cost of maintaining indexes during write operations.

Testing confirmed that approved queries were accepted, unsupported or malformed queries were rejected, the compound index was created, and the original dashboard behavior was preserved. Water Rescue returned River and Bay, Mountain or Wilderness Rescue returned Summit, Aspen, Snow, and Justice, and Disaster or Individual Tracking returned Summit, Tracker, Scout, and Justice. Reset displayed the complete representative dataset, and the table, pie chart, and map updated correctly when the selected filter changed.

Overall, the enhancement improved the artifact by strengthening its database-access rules, protecting destructive operations, removing embedded credentials, adding a purposeful index, and preserving the original dashboard functionality.